

Multiple Choice

1. **Answer:** B

Options: A) `sort('ab')`; B) `sorted('ab')`; C) `"ab".sort()`; D) `1/0`

Note: The expression `'sorted('ab')'` is valid in Python 3.5 because the `'sorted()'` function can take any iterable as an argument and returns a new list containing all items from the iterable in ascending order.

2. **Answer:** B

Options: A) A device driver is assigned to the device.; B) An Internet Protocol (IP) address is assigned to the device.; C) A packet number is assigned to the device.; D) A Web site is assigned to the device.

Note: When a new device is connected to the Internet, it is assigned an Internet Protocol (IP) address, which uniquely identifies it on the network and allows it to communicate with other devices.

3. **Answer:** C

Options: A) 4; B) 1; C) 10; D) 8

Note: The correct answer is 10 because the `sum` function in Python 3 computes the total of all the elements in the list `l`, which are 1, 2, 3, and 4, yielding a total of 10.

4. **Answer:** B

Options: A) `a`; B) `Chemistry`; C) `0`; D) `1`

Note: In Python, negative indexing allows you to access elements from the end of a list, so the expression `['a', 'Chemistry', 0, 1][-3]` retrieves the element at the third position from the end, which is `'Chemistry'`.

5. **Answer:** D

Options: A) A function that averages numeric values in a column or row; B) A function that counts the values in a column or row; C) A function that rounds a numeric value; D) A function that sorts values in a column or row

Note: A function that sorts values in a column or row allows users to easily identify outliers or anomalies, such as improbably high or low values, which may indicate data entry errors.

True / False

6. **Answer:** False

Options: A) True; B) False

Note: The statement is false because using `b[::2]` on the list `[11,13,15,17,19,21]` actually returns `[11, 15, 19]`, as the slicing syntax `b[start:end:step]` selects every second element starting from index 0.

7. **Answer:** False

Options: A) True; B) False

Note: The statement is false because $O(\log n)$ represents logarithmic growth, which increases at a much slower rate than $O(n^2)$, which represents quadratic growth, as the input size n becomes large.

8. **Answer:** False

Options: A) True; B) False

Note: The `random()` function in Python 3 generates a random float between 0.0 and 1.0 but does not set the integer starting value; instead, the `seed()` function is used to set the seed for the random number generator.

9. **Answer:** True

Options: A) True; B) False

Note: Shifting the number 1 to the left by 3 bits is equivalent to multiplying it by 2 to the power of 3, which results in $1 \times 2^3 = 8$.

10. **Answer:** True

Options: A) True; B) False

Note: The `strip()` function in Python 3 is designed to remove all types of leading and trailing whitespace characters, including spaces, tabs, and newline characters, from a string.

Long Form Response

11. **Answer:** The `'isupper()'` function in Python 3 checks whether all the characters in a string are uppercase letters. When called on a string, it returns `'True'` if every character is an uppercase letter and there is at least one character; otherwise, it returns `'False'`. For example, `'HELLO'.isupper()` returns `'True'`, while `'Hello'.isupper()` returns `'False'` because the letter "H" is uppercase but "e" is not. This function can be useful in applications such as validating user input, ensuring that certain data formats meet specific criteria, or formatting strings for display. Understanding how to use `'isupper()'` helps students learn about string manipulation in Python.

Note: correct because it accurately describes the functionality of the `'isupper()'` function in Python 3, including its return values based on the case of characters in a string and provides examples that demonstrate its usage.

12. **Answer:** The lambda function `r = lambda q: q * 2` in Python 3 creates an anonymous function that takes one input, `q`, and returns the result of `q` multiplied by 2. When we call `r(3)`, the function takes 3 as the input for `q`, resulting in `r(3) = 3 * 2`, which equals 6. This output demonstrates that lambda functions are useful for creating small, one-time use functions quickly without formally defining them using the `def` keyword. They are especially handy for short operations, allowing for more concise and readable code in situations such as filtering or mapping data.

Note: correct because it accurately describes how the lambda function defines a concise,

anonymous function that multiplies its input by 2, and it correctly computes the output for the given input, illustrating the utility of lambda functions for simple operations in Python.

End of Answer Key